

Relatório Técnico — Padrões Criacionais: Singleton & Factory Method

Disciplina: SI405 — Engenharia de Software
Instituição: Faculdade de Tecnologia — Unicamp
Autor: Bruno César de Souza Ferreira França
RA: 167955
Data: 07 de julho de 2026
Repositório: github.com/thebrunofranca/SI405-AP5

1. Análise dos Problemas Arquiteturais Identificados

1.1 Dispersão e duplicação da lógica de criação de geradores

O código original possuía lógica de criação de geradores de relatório duplicada em dois serviços distintos (`RelatorioService` e `ExportacaoService`). Ambas as classes continham blocos `if-else` ou `switch-case` equivalentes para instanciar os geradores concretos (`PdfRelatorioGenerator`, `CsvRelatorioGenerator`, `JsonRelatorioGenerator`):

```
// Em RelatorioService.java
if (formato == FormatoRelatorio.PDF) {
    PdfRelatorioGenerator generator = new PdfRelatorioGenerator();
    return generator.gerar(relatorio);
} else if (formato == FormatoRelatorio.CSV) {
    ...
}

// Em ExportacaoService.java – lógica idêntica repetida
switch (formato) {
    case PDF:
        PdfRelatorioGenerator pdfGenerator = new PdfRelatorioGenerator();
        ...
}
```

Isso viola o princípio DRY (*Don't Repeat Yourself*) e o princípio Open/Closed: ao adicionar um novo formato, seria necessário modificar múltiplas classes simultaneamente.

1.2 Acoplamento forte entre serviços e implementações concretas

`RelatorioService` e `ExportacaoService` dependiam diretamente das classes concretas dos geradores (`PdfRelatorioGenerator`, `CsvRelatorioGenerator`, `JsonRelatorioGenerator`). Não havia

abstração (interface ou classe abstrata) que representasse o conceito genérico de "gerador de relatório". Isso impede a substituição ou extensão dos geradores sem modificar os serviços.

1.3 Ausência de abstração para os geradores

Os três geradores existentes (`PdfRelatorioGenerator`, `CsvRelatorioGenerator`, `JsonRelatorioGenerator`) não compartilhavam nenhum contrato comum. O método `gerar(Relatorio)` existia em cada um de forma isolada, mas sem uma interface unificadora que permitisse tratá-los polimorficamente.

1.4 Configuração global inconsistente e fragmentada

Cada serviço instanciava seu próprio objeto `ConfiguracaoSistema` com valores diferentes e independentes:

Classe	nomeEmpresa	ambiente	diretorio	debug
<code>RelatorioService</code>	"Empresa XPTO"	"DEV"	"/tmp/relatorios"	false
<code>ExportacaoService</code>	"Empresa XPTO Ltda."	"PROD"	"/var/exports/relatorios"	false
<code>ConfiguracaoService</code>	"Empresa XPTO Ltda."	"DEV"	"/tmp/relatorios"	true

Essa inconsistência significa que diferentes partes do sistema operavam com visões diferentes da configuração. O ambiente exibido ao usuário ("DEV") era diferente do usado na exportação ("PROD"), e o debug estava ativo apenas em `ConfiguracaoService`. Isso é um bug arquitetural real: a configuração exibida não refletia a configuração efetivamente usada.

2. Justificativa das Decisões Arquiteturais Adotadas

2.1 Introdução da interface `RelatorioGenerator`

A criação da interface `RelatorioGenerator` com o método `gerar(Relatorio): String` estabelece um contrato comum para todos os geradores. Isso permite que os serviços trabalhem com a abstração, não com implementações concretas — princípio da Inversão de Dependência (DIP).

2.2 Criação do pacote `factory`

A lógica de decisão sobre qual gerador instanciar foi centralizada em `RelatorioGeneratorFactory`. Essa decisão separa claramente a responsabilidade de criação da responsabilidade de uso, tornando o sistema coeso. A adição de um novo formato requer apenas: (a) criar uma nova classe de gerador, (b) adicionar o valor ao enum `FormatoRelatorio`, e (c) adicionar um `case` na factory — sem tocar nos serviços existentes.

2.3 Aplicação do Singleton a `ConfiguracaoService`

`ConfiguracaoService` é o serviço natural para gerenciar a configuração global da aplicação. Ao torná-lo um Singleton, garante-se que todas as partes do sistema compartilhem exatamente a mesma instância de `ConfiguracaoSistema`, eliminando as inconsistências identificadas.

A implementação usa o padrão **Initialization-on-Demand Holder** (Bill Pugh Singleton), que é thread-safe sem a sobrecarga de `synchronized`:

```
private static class Holder {
    private static final ConfiguracaoService INSTANCIA = new ConfiguracaoService();
}
public static ConfiguracaoService getInstance() {
    return Holder.INSTANCIA;
}
```

A classe `ConfiguracaoSistema` foi mantida como um POJO mutável (sem Singleton) para preservar os testes unitários existentes que validam seu comportamento como objeto de domínio independente.

3. Explicação da Aplicação do Factory Method

Estrutura implementada

```
<>
RelatorioGenerator
+ gerar(Relatorio): String
    ^
    |
+-----+-----+-----+-----+
|         |         |         |         |
Pdf...   Csv...   Json...   Xml...   Html...
Generator Generator Generator Generator Generator

RelatorioGeneratorFactory
+ criar(FormatoRelatorio): RelatorioGenerator ← factory method
```

Funcionamento

O método estático `RelatorioGeneratorFactory.criar(FormatoRelatorio)` encapsula a decisão de qual implementação concreta instanciar. Os serviços que precisam gerar relatórios chamam apenas:

```
RelatorioGenerator generator = RelatorioGeneratorFactory.criar(formato);
String resultado = generator.gerar(relatorio);
```

Eles não conhecem `PdfRelatorioGenerator`, `XmlRelatorioGenerator` etc. — dependem apenas da abstração `RelatorioGenerator`.

Extensão para novos formatos (XML e HTML)

Para adicionar suporte a XML e HTML:

- Criadas as classes `XmlRelatorioGenerator` e `HtmlRelatorioGenerator`, ambas implementando `RelatorioGenerator`
- Adicionados os valores `XML` e `HTML` ao enum `FormatoRelatorio`
- Adicionados os `case` correspondentes em `RelatorioGeneratorFactory`
- `RelatorioService`, `ExportacaoService` e os testes existentes que iteram sobre `FormatoRelatorio.values()` passaram a suportar os novos formatos **sem qualquer modificação**

Isso demonstra o princípio Open/Closed em ação: o sistema foi aberto para extensão e fechado para modificação.

4. Explicação da Aplicação do Singleton

Problema resolvido

Antes da refatoração, três instâncias independentes de `ConfiguracaoSistema` coexistiam com valores contraditórios. O `ExportacaoService` usava "PROD" como ambiente, enquanto o `ConfiguracaoService` exibia "DEV" ao usuário.

Implementação

`ConfiguracaoService` foi refatorado com construtor privado e método de acesso estático:

```
public class ConfiguracaoService {  
  
    private static class Holder {  
        private static final ConfiguracaoService INSTANCIA = new ConfiguracaoService();  
    }  
  
    private final ConfiguracaoSistema configuracao = new ConfiguracaoSistema(  
        "Empresa XPTO Ltda.", "DEV", "/tmp/relatorios", true  
    );  
  
    private ConfiguracaoService() {}  
  
    public static ConfiguracaoService getInstance() {  
        return Holder.INSTANCIA;  
    }  
    ...  
}
```

Impacto

- `RelatorioService` e `ExportacaoService` passaram a usar `ConfiguracaoService.getInstance().getConfiguracao()`, garantindo que ambos trabalhem com a mesma configuração
- `Main.java` passou a usar `ConfiguracaoService.getInstance()` em vez de `new ConfiguracaoService()`
- A configuração exibida ao usuário é exatamente a mesma usada internamente pelos serviços

Garantia de thread-safety

O padrão Holder é thread-safe por garantia da especificação da JVM: a classe interna `Holder` só é inicializada na primeira chamada a `getInstance()`, e a JVM garante que essa inicialização é atômica, sem necessidade de `synchronized`.

5. Impactos da Refatoração na Organização e Extensibilidade do Sistema

5.1 Organização

Aspecto	Antes	Depois
Pacotes	<code>generator</code> , <code>service</code> , <code>domain</code>	Adicionado pacote <code>factory</code>
Responsabilidade	Serviços criavam e usavam geradores	Factory cria; serviços apenas usam via interface
Configuração	Fragmentada em 3 classes com valores distintos	Centralizada em 1 Singleton consistente
Contrato de geradores	Inexistente (duck typing)	Interface <code>RelatorioGenerator</code> bem definida

5.2 Extensibilidade

Adicionar um novo formato de relatório (ex: MARKDOWN):

Antes: modificar `RelatorioService` (bloco if-else), modificar `ExportacaoService` (bloco switch), criar a classe do gerador — 3 arquivos alterados, risco de regressão.

Depois: criar `MarkdownRelatorioGenerator` implements `RelatorioGenerator`, adicionar `MARKDOWN` ao enum, adicionar `case MARKDOWN` na factory — os serviços existentes não precisam ser tocados.

5.3 Testabilidade

- Os testes de serviço (`RelatorioServiceTest` , `ExportacaoServiceTest`) continuam funcionando sem modificação
- Os testes que iteravam sobre `FormatoRelatorio.values()` automaticamente cobrem XML e HTML sem alteração
- Novos testes específicos para factory (`RelatorioGeneratorFactoryTest`) validam o comportamento de criação de forma isolada
- O teste do Singleton (`singletonDeveRetornarMesmaInstancia`) verifica a propriedade fundamental da instância única

5.4 Resumo de métricas

Métrica	Valor
Arquivos criados	6
Arquivos modificados	8
Testes antes da refatoração	23
Testes após a refatoração	34
Testes com falha	0
Novos formatos suportados	2 (XML, HTML)